



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-49975 Apache HTTP Server: mod_http2 denial of service

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-12
Version	1.0
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-49975 Apache HTTP Server: mod_http2 denial of service - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-12 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

CVE-2026-49975 Apache HTTP Server: mod_http2 denial of service

1. Executive Summary

CVE-2026-49975 is a remotely reachable denial-of-service vulnerability affecting Apache HTTP Server deployments that expose HTTP/2 via `mod_http2`. Apache's CNA record describes the issue as a CWE-789 memory-allocation flaw that lets malicious HTTP requests force excessive memory consumption and service instability. Apache marks the issue as moderate severity, while NVD normalizes it to CVSS v3.1 7.5 High (`AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`) because the impact is unauthenticated, network-reachable, and availability-focused.

The strongest public technical context comes from Apache's own fix history and from contemporaneous HTTP/2 Bomb research published by Calif.IO. In Apache's case, the exploit path abuses HTTP/2 cookie-crumb handling so repeated cookie header fragments do not count effectively against `LimitRequestFields`, while the server repeatedly rebuilds merged cookie state in memory. Public research demonstrated roughly 4,000:1 memory amplification against Apache httpd 2.4.67 and showed that a single client could pin approximately 32 GB of memory in under 20 seconds under lab conditions.

For defenders, this is primarily an exposure-management and resilience problem. Internet-facing Apache systems that terminate HTTP/2 for web applications, APIs, portals, reverse proxies, and partner services are the highest-priority patch targets. The durable fix is Apache HTTP Server 2.4.68 or a backported vendor package carrying the `mod_http2` remediation. If patching cannot occur immediately, disable HTTP/2 for exposed listeners or place the origin behind a control point that enforces strict HTTP/2 header-count limits and aggressive memory / worker isolation.

2. Intelligence Requirement

This report addresses the following intelligence requirement: what does CVE-2026-49975 change for defenders operating Apache HTTP Server, which systems are most exposed, how credible is the public exploitation narrative, and what immediate remediation, detection, and hunting actions should be prioritized?

Operational questions answered in this review are summarized below.

Question	Assessment
What is the vulnerability?	An HTTP/2 request-handling denial-of-service condition in Apache HTTP Server that can force excessive memory allocation and service degradation or outage.
Which Apache versions are affected?	Apache HTTP Server 2.4.17 through 2.4.67 according to Apache CNA and Apache security advisories.
What is the fixed version?	Apache HTTP Server 2.4.68, with upstream <code>mod_http2</code> fix lineage tied to <code>mod_http2</code> v2.0.41 and Apache revision <code>r1934882</code> .
Is exploitation public?	Yes. Public technical writeups, fix diffs, and proof-of-concept discussion exist.
Is active exploitation confirmed by CISA KEV?	No KEV listing was present when checked on 2026-06-12.
Which environments matter most?	Internet-facing Apache deployments with HTTP/2 enabled, especially reverse proxies and application front ends that can be reached pre-authentication.

3. Source Review and Confidence

Source	Contribution	Confidence
Apache CVE JSON / Apache security advisory	Canonical affected-version range, severity text, timeline, release target, upstream fix	High

	lineage	
CVE Program API record	Publication dates, canonical CNA description, credits, references	High
NVD API entry	CVSS vector, severity normalization, CPE-style affected-product expression	High
OSS-security / Openwall posts	Disclosure timeline and vendor restatement of impact	High
Calif.IO "HTTP/2 Bomb" research	Public exploitation mechanics, amplification estimates, temporary mitigations, multi-vendor context	Medium
Microsoft Security Update Guide intake item	Discovery / intake source only	Medium

Assessment confidence is high for the core defensive facts: affected Apache versions, availability impact, public disclosure status, and the availability of a fixed Apache release. Confidence is medium for exploit-prevalence claims because the public record shows credible weaponization and public proof-of-concept detail, but not a confirmed broad campaign or CISA KEV inclusion at review time.

A small source-quality caveat is worth noting: Apache's description text says `mod\_http`, while the title, fix commit, and public technical analysis all point to `mod\_http2` request processing and cookie-merging behavior. Defenders should anchor on the fix path rather than on the wording discrepancy: the authoritative remediation is the Apache HTTP Server 2.4.68 / `mod\_http2` update path.

4.1 Vulnerability metadata

Field	Value
CVE	CVE-2026-49975
Vendor	Apache Software Foundation
Product	Apache HTTP Server
Affected versions	2.4.17 through 2.4.67
Fixed version	2.4.68
Component context	`mod_http2` / HTTP/2 request handling
Weakness	CWE-789 Memory Allocation with Excessive Size Value
Apache severity text	Moderate
NVD severity	CVSS v3.1 7.5 High
Attack vector	Network
Privileges required	None
User interaction	None
Security outcome	Denial of service / memory pressure / service instability
KEV status checked 2026-06-12	Not listed

--	--

4.2 Exploit mechanics and defender implications

Public research and the upstream Apache fix indicate that the attack path abuses HTTP/2 header behavior rather than traditional volumetric flooding. The exploit combines HPACK amplification with HTTP/2 stream stalling so the attacker pays very little wire cost while the server holds a large amount of state in memory.

For Apache specifically, the key issue is how split HTTP/2 cookie crumbs are merged. RFC 9113 allows a client to send multiple `Cookie` header fragments. Before the fix, Apache's `mod\_http2` logic merged repeated cookie crumbs without properly treating each crumb as an additional header for `LimitRequestFields` enforcement. The upstream fix sets the merge operation to count as an add and ignores duplicate empty cookie headers, closing the bypass condition described in public research.

4.3 Exposure profile

Exposure pattern	Why it matters	Priority
Internet-facing Apache with HTTP/2 enabled	Reachable pre-authentication by low-cost remote requests	Immediate
Apache reverse proxies and API front ends	Concentrates application availability risk and multi-tenant blast radius	Immediate
Systems behind CDN / WAF / L7 proxy	Risk may be reduced if the fronting layer enforces safer HTTP/2 parsing and limits	High
Internal-only Apache services	Lower exposure to opportunistic internet scanning, but still patch for insider or lateral abuse	High
HTTP/1.1-only Apache listeners	Not on the primary exploit path described for this CVE	Reduced

4.4 Disclosure and patch timeline

Date	Event
2026-05-26	Reported to Apache security team
2026-05-27	Upstream `mod_http2` fix published (`35c6e405390ed361189a82acd96675401ea5947c`)
2026-06-02	Apache 2.4.x fix tracked as revision `r1934882`
2026-06-08	Apache HTTP Server 2.4.68 released and CVE published

5. Threat Context

This vulnerability sits inside a broader June 2026 public disclosure wave around the so-called HTTP/2 Bomb technique. Apache is not the only server family discussed in that research, but CVE-2026-49975 is the Apache-specific tracking identifier and should be handled as an Apache availability risk rather than as a generic protocol panic.

There is no strong public evidence, at review time, of a named threat actor or mass exploitation campaign focused specifically on Apache CVE-2026-49975. That said, the practical barriers to abuse appear low: the issue is remotely reachable, the fix diff is public, the research writeup is detailed, and the attack is meaningful even when the goal is only service degradation rather than full compromise.

Organizations should treat this as a high-priority operational risk when Apache is part of externally reachable customer, partner, login, API, or reverse-proxy infrastructure. The likely abuse scenarios are opportunistic internet scanning, nuisance disruption, extortion-style service pressure, or use during broader intrusion activity to distract defenders and degrade service availability.

5.1 Representative attack scenarios

Scenario	Description	Defensive meaning
Opportunistic internet disruption	External actor sends crafted HTTP/2 requests to exposed Apache endpoints until workers or hosts enter memory pressure and slow drastically	Patch fast and restrict HTTP/2 exposure
Targeted business-hour outage	Adversary times requests against critical customer or partner portals to maximize visible downtime	Prioritize internet-facing business services and response playbooks
Chained operational pressure	Attacker combines low-cost DoS with separate intrusion or phishing activity to create alert fatigue	Correlate web instability with broader incident indicators

5.2 MITRE ATT&CK context

Tactic	Technique	Relevance
Impact	T1499 Endpoint Denial of Service	Best fit for the publicly described availability outcome
Initial Access / Exploit context	T1190 Exploit Public-Facing Application	Applicable when an exposed Apache HTTP/2 listener is abused directly from the internet
Reconnaissance	T1595 Active Scanning	Likely precursor for identifying exposed HTTP/2-capable Apache systems

6. Detection and Hunting

Direct signature detection is harder than for classic exploit traffic because the attack relies on protocol-valid HTTP/2 behaviors and resource retention. Detection should therefore combine exposure discovery, anomalous request telemetry, and host-side instability signals.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal
Apache listeners with HTTP/2 enabled	Config management, package inventory, web server configs	Identify patch / mitigation scope
Sudden Apache worker memory growth or OOM activity	Host telemetry, EDR, kernel / system logs	Surface successful or near-successful exploitation
Spikes in HTTP/2 requests from a small set of clients	Reverse proxy, WAF, load balancer, network telemetry	Find likely attack origin or rehearsal activity
Repeated Apache restarts, crashes, or prolonged latency during modest inbound volume	Service monitoring, APM, EDR	Detect service degradation without classic bandwidth spikes

Requests with suspiciously high cookie-fragment counts where header detail is logged	WAF / proxy logs with header visibility	Closest protocol-specific detection opportunity
--	---	---

6.2 Detection engineering notes

- Prioritize sensors in front of Apache, because origin access logs often do not preserve enough HTTP/2 header structure to count cookie crumbs directly.
- If a load balancer or reverse proxy can enforce maximum header count per request, treat that as both mitigation and a high-value logging point.
- Host-based observables matter: memory pressure, swap growth, worker churn, process restarts, and OOM kills are often the earliest reliable signs.

6.3 KQL hunt for Apache exposure plus instability

```

let lookback = 14d;
let apache_hosts = DeviceProcessEvents
| where Timestamp > ago(lookback)
| where FileName in~ ("httpd", "apache2") or ProcessCommandLine has_any ("mod_http2", "Protocols h2", "apache2", "httpd")
| summarize FirstSeen=min(Timestamp), LastSeen=max(Timestamp), SampleCmd=any(ProcessCommandLine) by DeviceName;
let apache_pressure = DeviceEvents
| where Timestamp > ago(lookback)
| where ActionType has_any ("ApplicationCrash", "ServiceStopped", "ServiceCrash", "LowMemory", "OomKill")
| where FileName in~ ("httpd", "apache2") or InitiatingProcessFileName in~ ("httpd", "apache2")
| project Timestamp, DeviceName, ActionType, FileName, InitiatingProcessFileName, ReportId;
let apache_net = DeviceNetworkEvents
| where Timestamp > ago(lookback)
| where LocalPort in (80, 443, 8080, 8443)
| where InitiatingProcessFileName in~ ("httpd", "apache2")
| summarize Connections=count(), DistinctRemoteIPs=dcount(RemoteIP), SampleRemoteIPs=make_set(RemoteIP, 20) by DeviceName, bin(Timestamp, 15m);
apache_hosts
| join kind=leftouter apache_pressure on DeviceName
| join kind=leftouter apache_net on DeviceName
| order by Timestamp desc nulls last

```

6.4 Splunk hunt for Apache instability around HTTP/2 traffic

```

(index=os OR index=linux OR index=apache)
(
  (sourcetype=apache:error ("AH" OR "mod_http2" OR "server reached MaxRequestWorkers" OR "out of memory"))
  OR (sourcetype=syslog ("oom-kill" OR "Out of memory" OR "Killed process" OR "httpd" OR "apache2"))
)
| eval host_asset=coalesce(host, dest, dvc)
| stats count values(sourcetype) as sourcetypes values(_raw) as samples by host_asset
| join type=left host_asset [ search index=netfw OR index=proxy OR index=waf earliest=-24h
  ("HTTP/2" OR http_version="2" OR alpn="h2")
  | stats count as h2_events dc(src_ip) as distinct_src values(src_ip) as sample_src by host_asset ]
| sort - h2_events - count

```

6.5 Threat-hunting hypotheses

Hypothesis	What to prove or disprove
Exposed Apache systems are still running vulnerable builds	Inventory plus package validation should confirm 2.4.68 or vendor backport status
Apache memory pressure incidents since 2026-06-02 may reflect attempted exploitation	Correlate OOM / restart events with inbound HTTP/2 activity

Fronting infrastructure may already be mitigating the issue without origin visibility	Review WAF / proxy header-limit and HTTP/2 normalization policies
Attack rehearsal may appear as low-bandwidth but high-frequency traffic from a small set of sources	Hunt for repeated HTTP/2 requests followed by worker instability rather than bandwidth spikes

7. Recommendations

Priority	Owner	Action	Rationale
Immediate	Web / Platform Engineering	Upgrade Apache HTTP Server to 2.4.68 or apply a vendor backport that includes the `mod_http2` fix	Primary remediation path
Immediate	Web / Platform Engineering	Disable HTTP/2 on exposed Apache listeners if patching cannot be completed immediately	Most reliable temporary mitigation
High	Security Engineering / Network	Enforce strict header-count and request-normalization limits at reverse proxies, WAFs, or edge services terminating HTTP/2	Reduces exploit reach to origin
High	SRE / Operations	Alert on Apache worker crashes, OOM kills, swap growth, and sudden service restart clusters	Detects active disruption quickly
High	Asset / Vulnerability Management	Inventory every internet-facing Apache deployment and verify whether `mod_http2` is enabled	Scoping is as important as patching
Medium	Incident Response	Preserve logs, memory-pressure indicators, and front-end telemetry if unexplained Apache instability is observed	Supports attribution and confirmation
Medium	Architecture / Resilience	Apply worker memory caps, container limits, or cgroup protections where feasible	Improves failure mode when exploitation occurs

Recommended execution order:

1. Identify all Apache listeners that terminate HTTP/2.
2. Patch to 2.4.68 or confirm vendor backport coverage.
3. Disable HTTP/2 where patching is delayed or where the service does not require it.
4. Review edge-control enforcement for header count limits and HTTP/2 normalization.
5. Hunt retrospectively for memory-pressure and restart anomalies from 2026-06-02 onward.

8. References

- Microsoft Security Update Guide intake item, `CVE-2026-49975`
- <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-49975>
- Apache HTTP Server security advisory, `Apache HTTP Server 2.4 vulnerabilities`

- https://httpd.apache.org/security/vulnerabilities_24.html
- Apache CVE JSON record, `CVE-2026-49975`
- <https://httpd.apache.org/security/json/CVE-2026-49975.json>
- CVE Program API record, `CVE-2026-49975`
- <https://cveawg.mitre.org/api/cve/CVE-2026-49975>
- NVD CVE 2.0 API, `CVE-2026-49975`
- <https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-49975>
- OSS-security / Openwall disclosure summary, `HTTP/2 Bomb affects Apache httpd, nginx, envoy, & pingora`
- <http://www.openwall.com/lists/oss-security/2026/06/03/3>
- OSS-security / Openwall Apache CVE post, `CVE-2026-49975: Apache HTTP Server: mod\_http2 denial of service`
- <http://www.openwall.com/lists/oss-security/2026/06/08/16>
- Calif.IO research, `Codex Discovered a Hidden HTTP/2 Bomb`
- <https://blog.calif.io/p/codex-discovered-a-hidden-http2-bomb>
- Upstream `mod\_http2` fix commit, `cookie request header counting`
- https://github.com/icing/mod_h2/commit/35c6e405390ed361189a82acd96675401ea5947c